```
function
ps_mrm(filenames_slc,slc_selection,psc_selection_gridsize,calfactors,crop,crop_i
n)

%
% ----------------------------------------------------------------------
% File.............: ps_mrm.m
% Version & Date..: 1.7.2.16, 12-DEC-2009
% Authors.........: Freek van Leijen
%                   Gini Ketelaar
%                   Delft Institute of Earth Observation and Space Systems
%                   Delft University of Technology
% ----------------------------------------------------------------------
%
% This software is developed by Delft University of Technology and is
% intended for scientific use only. Applications for commercial use are
% prohibited.
%
% Copyright (c) 2004-2009 Delft University of Technology, The Netherlands
%
% Change log
% v1.7.2.16, Freek van Leijen
% - filenames in cells
% - separate calibration
% vsvn, Freek van Leijen
% - output of amplitude dispersion
%


% ----------------------------------------------------------------------
% Initialize
% ----------------------------------------------------------------------

global max_mem_buffer Nlines Npixels az_spacing r_spacing project_id

Nslc = size(filenames_slc,1);
Nslc_selected = length(slc_selection);
Nifgs = Nslc-1;


% ----------------------------------------------------------------------
% Open the data files
% ----------------------------------------------------------------------

mrm_fid = fopen([project_id '_mrm.raw'],'w');
% file with multi reflectivity map
amp_disp_fid = fopen([project_id '_amp_disp.raw'],'w');
% file with amplitude dispersion


% ----------------------------------------------------------------------
% Determine buffersize
% ----------------------------------------------------------------------

Nlines_g = round(psc_selection_gridsize/az_spacing);
Npixels_g = round(psc_selection_gridsize/r_spacing);

Ng_az = floor(Nlines/Nlines_g);
Ng_r = floor(Npixels/Npixels_g);
offset_az = floor(rem(Nlines,Nlines_g)/2);
offset_r = floor(rem(Npixels,Npixels_g)/2);

buffer_size_az = floor(max_mem_buffer/(8*8*Nifgs*Npixels*Nlines_g));
% buffer_size_az in number of gridcells in azimuth direction
```

```
% 6 because of 2 float array's, 2 complex array's, 1 multi image
% and 1 for the rest, 8 because of double

if (buffer_size_az == 0)
  buffer_size_az = 1;
  buffer_size_r = floor(max_mem_buffer/(8*8*Nifgs*Npixels_g*Nlines_g));
  if (buffer_size_r == 0)
    error('Not enough memory to proceed. Please increase the allocated memory or
reduce the grid size');
  end
  Nbuffers_r = floor(Ng_r/buffer_size_r);
  rem_buffer_r = rem(Ng_r,buffer_size_r);
else
  buffer_size_r = Ng_r;
  Nbuffers_r = 1;
  rem_buffer_r = 0;
end

if (buffer_size_az*Nlines_g > Nlines)
  buffer_size_az = Ng_az;
end

Nbuffers_az = floor(Ng_az/buffer_size_az);
rem_buffer_az = rem(Ng_az,buffer_size_az);



% ------------------------------------------------------------------------
% Determine borders gridcells
% ------------------------------------------------------------------------

grid_array_az = repmat(NaN,Ng_az,3);
grid_array_az(:,1) = [1 offset_az+Nlines_g+1:Nlines_g:offset_az+(Ng_az-
1)*Nlines_g+1]';
grid_array_az(:,2) = [offset_az+Nlines_g:Nlines_g:offset_az+ ...
                    (Ng_az-1)*Nlines_g Nlines]';
for v = 1:Nbuffers_az
  buffer = (v-1)*buffer_size_az+1:v*buffer_size_az;
  grid_array_az(buffer,3) = repmat(v,length(buffer),1);
end

if (rem_buffer_az ~= 0)
  buffer = v*buffer_size_az+1:Ng_az;
  grid_array_az(buffer,3) = repmat(v+1,length(buffer),1);
  Nbuffers_az = Nbuffers_az + 1; % add remaining buffer
end

grid_array_r = repmat(NaN,Ng_r,3);
grid_array_r(:,1) = [1 offset_r+Npixels_g+1:Npixels_g:offset_r+(Ng_r-
1)*Npixels_g+1]';
grid_array_r(:,2) = [offset_r+Npixels_g:Npixels_g:offset_r+ ...
                    (Ng_r-1)*Npixels_g Npixels]';

for v = 1:Nbuffers_r
  buffer = (v-1)*buffer_size_r+1:v*buffer_size_r;
  grid_array_r(buffer,3) = repmat(v,length(buffer),1);
end

if (rem_buffer_r ~= 0)
  buffer = v*buffer_size_r+1:Ng_r;
  grid_array_r(buffer,3) = repmat(v+1,length(buffer),1);
  Nbuffers_r = Nbuffers_r + 1; % add remaining buffer
end
```

```
% output to screen
fprintf(1,'\n');
fprintf(1,'Data devided in %3.0f buffers of %3.0f grid cells in azimuth
direction\n', ...
        Nbuffers_az,buffer_size_az);
if (rem_buffer_az ~= 0)
  fprintf(1,'plus a remaining buffer of %3.0f grid cells\n', ...
          rem_buffer_az);
end
fprintf(1,'and in %3.0f buffers of %3.0f grid cells in range direction\n', ...
        Nbuffers_r,buffer_size_r);
if (rem_buffer_az ~= 0)
  fprintf(1,'plus a remaining buffer of %3.0f grid cells\n', ...
          rem_buffer_r);
end
fprintf(1,'\n');



% ------------------------------------------------------------------------
% Main loop per buffer
% ------------------------------------------------------------------------

for v = 1:Nbuffers_az


  % ------------------------------------------------------------------------
  % Read data
  % ------------------------------------------------------------------------

  index_az = find(grid_array_az(:,3)==v);
  Ngrid_az = length(index_az);
  begin_buffer_az = grid_array_az(index_az(1),1);
  end_buffer_az = grid_array_az(index_az(end),2);
  buffer_size_az = end_buffer_az-begin_buffer_az+1;

  multi_refl_map = repmat(NaN,buffer_size_az,Npixels);
  amp_disp_map = repmat(NaN,buffer_size_az,Npixels);

  for w = 1:Nbuffers_r

    fprintf(1,'Processing buffer %3.0f in azimuth and buffer %3.0f in
range\n',v,w)

    index_r = find(grid_array_r(:,3)==w);
    Ngrid_r = length(index_r);
    begin_buffer_r = grid_array_r(index_r(1),1);
    end_buffer_r = grid_array_r(index_r(end),2);
    buffer_size_r = end_buffer_r-begin_buffer_r+1;

    slc_array = repmat(NaN,[buffer_size_az buffer_size_r Nslc]);

    for z = 1:Nifgs
      Nlines_file = crop_in(z,2)-crop_in(z,1)+1;
      loffset = crop(1)-crop_in(z,1);
      poffset = crop(3)-crop_in(z,3);
      slc_array(:,:,z) =
freadbk(char(filenames_slc(z)),Nlines_file,'cpxfloat32',...
            begin_buffer_az+loffset,end_buffer_az+loffset,...
            begin_buffer_r+poffset,end_buffer_r+poffset);
    end

    if Nslc == Nifgs+1
      % read last slc
```

```
      Nlines_file = crop_in(Nslc,2)-crop_in(Nslc,1)+1;
      loffset = crop(1)-crop_in(Nslc,1);
      poffset = crop(3)-crop_in(Nslc,3);

      fileinfo = dir(char(filenames_slc(Nslc)));
      Nel = (crop_in(Nslc,2)-crop_in(Nslc,1)+1)*(crop_in(Nslc,4)-
crop_in(Nslc,3)+1);
      if fileinfo.bytes==Nel*2*4
        master_format = 'cpxfloat32';
      elseif fileinfo.bytes==Nel*2*2;
        master_format = 'cpxint16';
      else
        error('Something is wrong with the file size of the master SLC.');
      end

      slc_array(:,:,Nslc) =
freadbk(char(filenames_slc(Nslc)),Nlines_file,master_format,...
            begin_buffer_az+loffset,end_buffer_az+loffset,...
            begin_buffer_r+poffset,end_buffer_r+poffset);
    end



    % ------------------------------------------------------------------------
    % Amplitude calibration
    % ------------------------------------------------------------------------

    amp_array = abs(slc_array);
    for k = 1:Nslc_selected
      amp_array(:,:,slc_selection(k)) =
amp_array(:,:,slc_selection(k))/calfactors(k);
    end
    mean_amp = nanmean(amp_array(:,:,slc_selection),3);
    multi_refl_map(:,begin_buffer_r:end_buffer_r) = mean_amp;

    std_amp = nanstd(amp_array(:,:,slc_selection),0,3);
    % the 0-flag causes std to normalize by (N-1)
    amp_disp = std_amp./mean_amp; % amplitude dispersion

    amp_disp_map(:,begin_buffer_r:end_buffer_r) = amp_disp;


    % ------------------------------------------------------------------------
    % Write the data to file
    % ------------------------------------------------------------------------

    if w == Nbuffers_r % complete in range direction

      multi_refl_map = log10(multi_refl_map);
      fwrite(mrm_fid,multi_refl_map','single'); % transpose is important!
      fwrite(amp_disp_fid,amp_disp_map','single');
    end

  end %for

end % end main loop



% ------------------------------------------------------------------------
% Close all open files
% ------------------------------------------------------------------------

fclose('all');
```

```
% ------------------------------------------------------------------------
% Save mrm as image
% ------------------------------------------------------------------------

Nlines_buffer = min(Nlines,floor(max_mem_buffer/(8*Npixels)));
Nlines_buffer_new = ceil(Nlines_buffer);
Nbuffer = ceil(Nlines/Nlines_buffer);
Nlines_rem_buffer = rem(Nlines,Nlines_buffer);

mrm_fid_in = fopen([project_id '_mrm.raw'],'r');
mrm_fid_out = fopen([project_id '_mrm_uint8.raw'],'w');
for v = 1:Nbuffer
  sl = (v-1)*Nlines_buffer_new+1;
  if (v==Nbuffer)&&(Nlines_rem_buffer~=0)
    Nlines_buffer = Nlines_rem_buffer;
    el = Nlines;
  else
    el = v*Nlines_buffer_new;
  end

  mrm = fread(mrm_fid_in,[Npixels Nlines_buffer],'*single')';

  mrm_min = min(mrm(:));
  mrm_max = max(mrm(:));
  mrm_new = uint8(255*(mrm-mrm_min)./(mrm_max-mrm_min));
  mrm_new = imadjust(mrm_new); %stretch

  fwrite(mrm_fid_out,mrm_new','uint8');
end
fclose(mrm_fid_in);
fclose(mrm_fid_out);
```